



No se trata solo de programar páginas, sino de crear soluciones que conecten al mundo. Hoy comenzamos a construir el futuro

Ing. Victor Hugo Perez Rojas

CEO-FOUNDER VAIXS

Especialista en Tecnologías de la Información

victorperez@vaixs.net

[linkedin.com/in/victor-hugo-perez-rojas-b412a92a](https://www.linkedin.com/in/victor-hugo-perez-rojas-b412a92a)





Tecnologías de Internet

Ing. Victor Hugo Perez Rojas

CEO-FOUNDER VAIXS

Especialista en Tecnologías de la Información

victorperez@vaixs.net

[linkedin.com/in/victor-hugo-perez-rojas-b412a92a](https://www.linkedin.com/in/victor-hugo-perez-rojas-b412a92a)



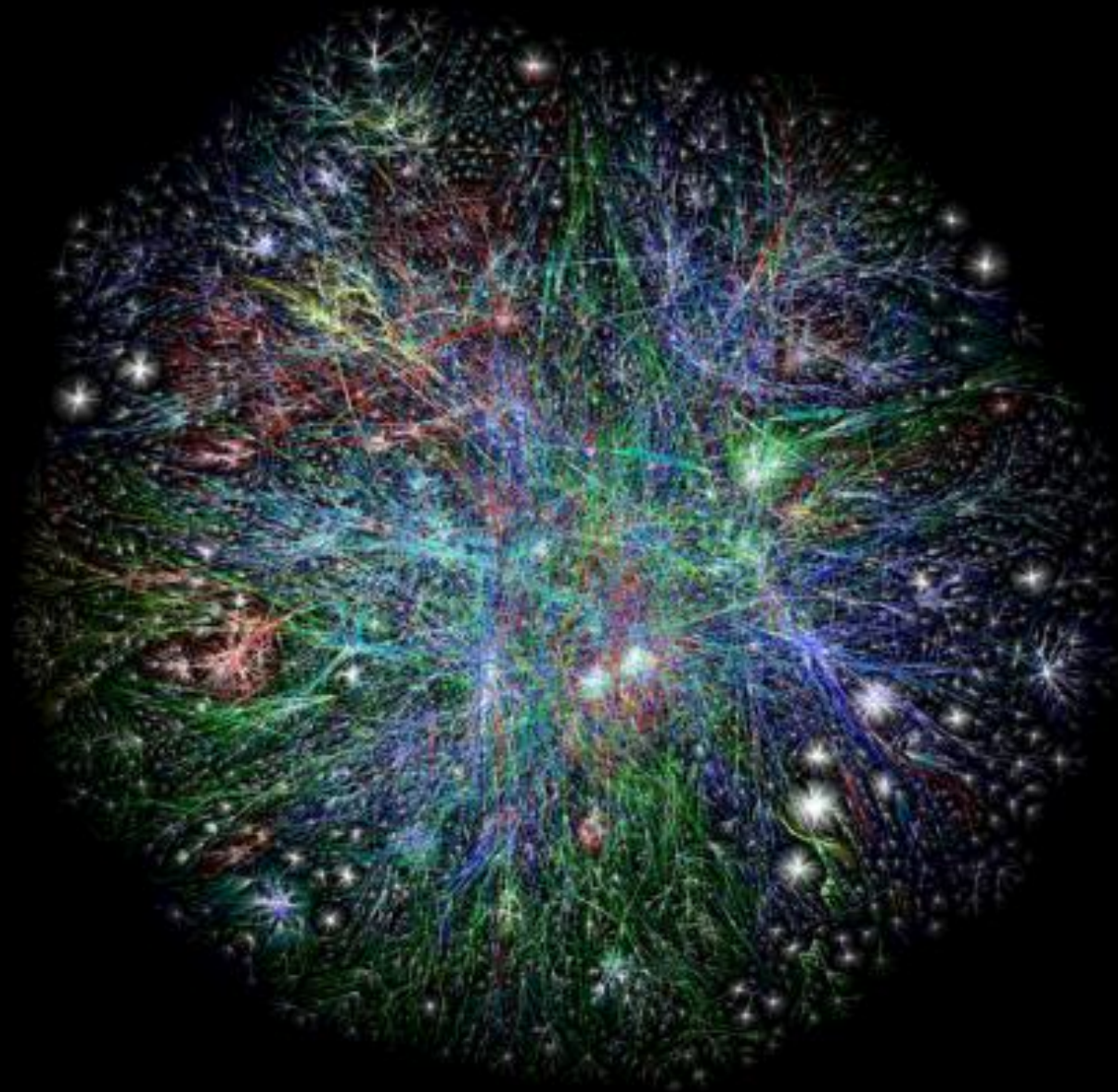
Evaluación

- **La evaluación es principalmente práctica: los trabajos, proyectos y laboratorios tendrán más peso que la teoría.**
- **La nota final es sobre 100 puntos.**
- **Los exámenes son los que tienen mayor valor dentro de la calificación.**
- **Para aprobar la materia, necesitas obtener más de 50 puntos.**
- **Si tu nota se encuentra entre 40 y 50 puntos, podrás acceder a segunda instancia.**

Actividad 1

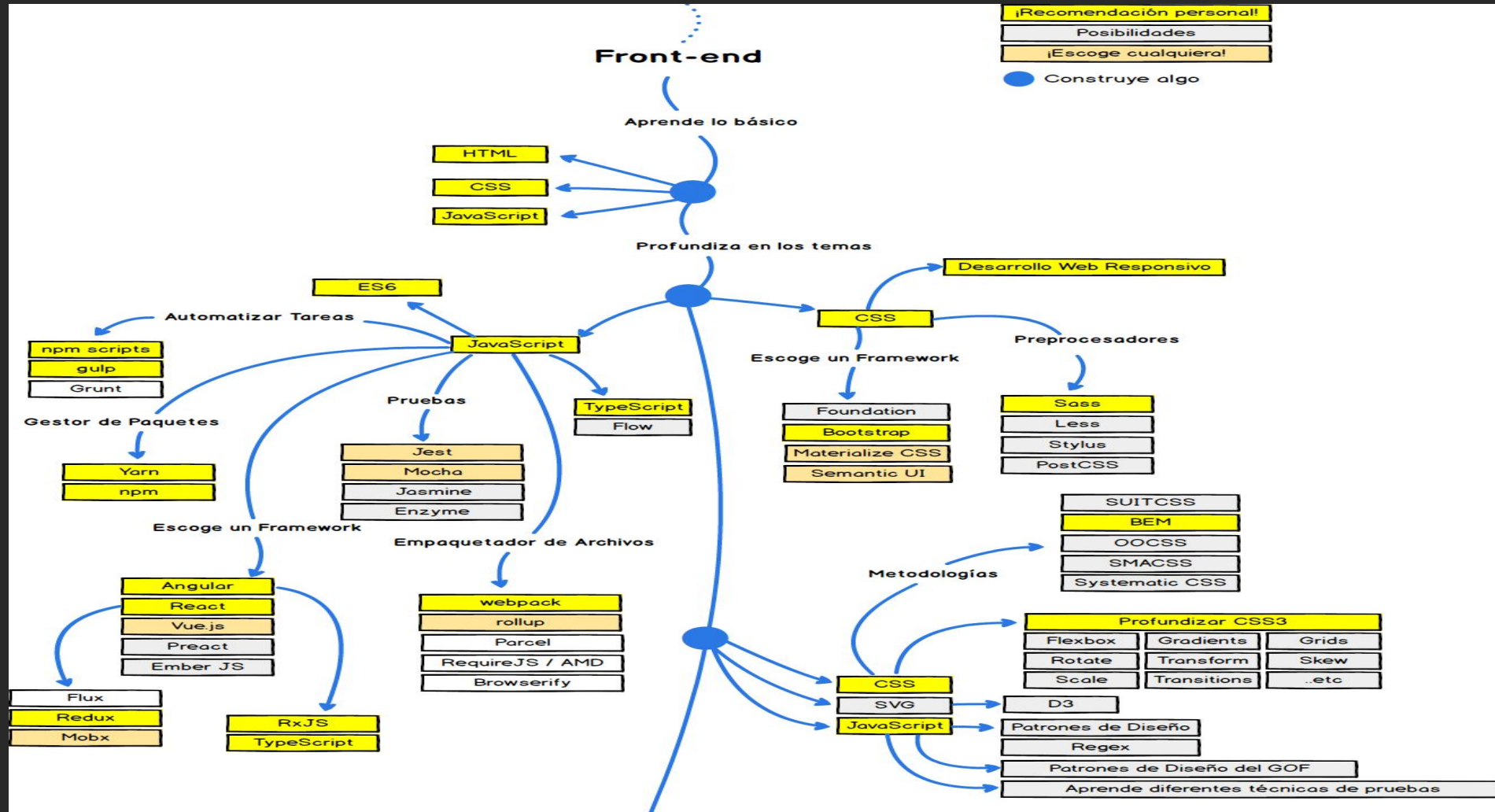
Actividad Diagnóstica:

Conócete y cuéntanos tu experiencia, intereses y expectativas en Ingeniería de Sistemas y Tecnologías de Internet.



<http://www.opte.org/maps/>

<https://roadmap.sh/backend>
<https://roadmap.sh/frontend>





Actividad 2

Instalación de herramientas

¿Qué es Internet?

- Red global de computadoras interconectadas
- Comunicación mediante protocolos estandarizados
- Nadie es dueño del Internet, es una red descentralizada

Direcciones IP y DNS

- **IP (Internet Protocol):** dirección única de cada dispositivo
- DNS (Domain Name System): traduce nombres como google.com en Ips

Ejemplo: Escribes → www.uab.edu.bo

DNS → 200.105.144.24 ?

Pregunta en clase: ¿Qué pasaría si no existiera DNS?

Paquetes, Ruteo y Confiabilidad

- Información se divide en paquetes
- Paquetes viajan por diferentes caminos hasta llegar
- Routers deciden la mejor ruta
- Protocolos de confiabilidad: TCP asegura que lleguen completos ?

Qué pasaría si Internet no dividiera la información en paquetes y siempre enviara todo de golpe?

HTTP y HTML

- HTTP (HyperText Transfer Protocol): cómo viajan las páginas web
- HTML (HyperText Markup Language): cómo se estructuran las páginas

Ejemplo:

Navegador solicita index.html

Servidor responde con la página

¿Qué diferencia hay entre HTTP y HTTPS?

Seguridad: Criptografía y Llaves Públicas

- Uso de SSL/TLS para cifrar la información
 - Garantiza confidencialidad y autenticidad
- Ejemplo: el candado  en el navegador

Ciberseguridad y Ciberdelitos

- Ciberseguridad: medidas de protección en la red

Ejemplos de ciberdelitos:

Phishing (correos falsos) Malware / Ransomware

Robo de identidad

¿Han recibido un correo sospechoso?

Preguntas de Reflexión

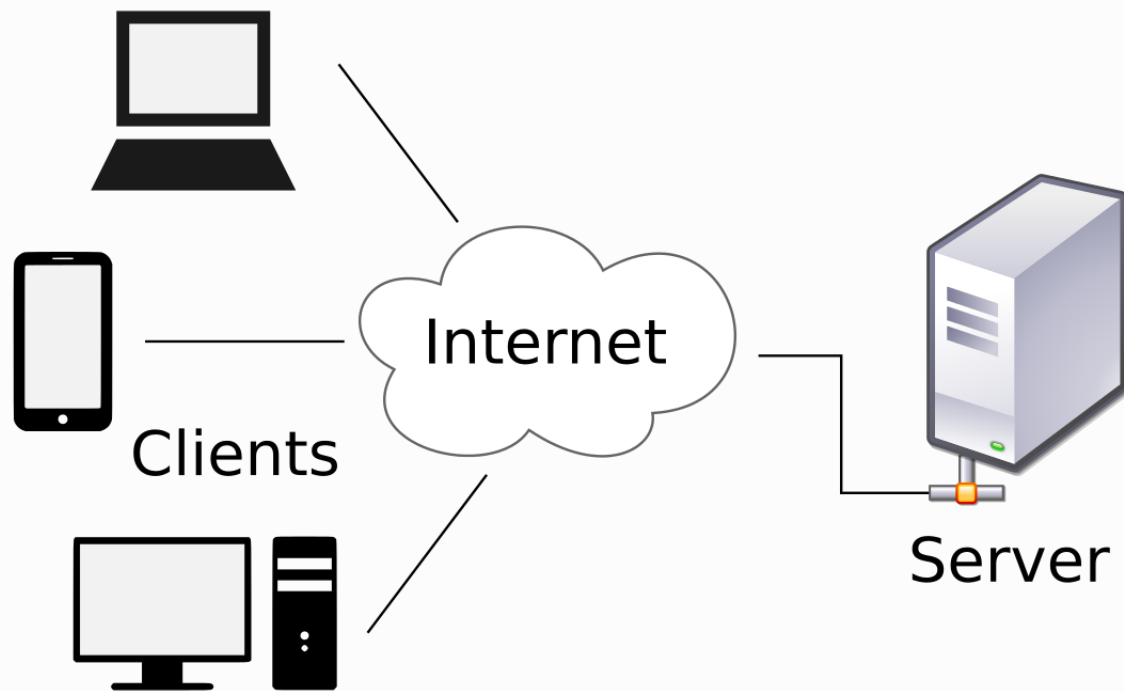
- ¿Quién “gobierna” Internet?
- ¿Por qué es importante el DNS?
- ¿Qué diferencia hay entre HTTP y HTTPS?
- ¿Qué riesgos existen al navegar sin seguridad?

Actividad 3

WireShark

Arquitectura cliente servidor

Es una aplicación de software basada en la arquitectura CLIENTE – SERVIDOR a la que los usuarios pueden acceder a través de un navegador web (cliente) siempre y cuando sea posible la conexión al servidor.



Internet es la forma de conexión mas común entre un cliente y un servidor, sin embargo si se utiliza un servidor en una red local, también se considera una aplicación web.

¿Qué es HTTP y HTTPS?

- HTTP: Protocolo que permite la comunicación entre cliente y servidor.
- HTTPS: HTTP con seguridad (cifrado TLS/SSL). Protege la información.

¿Qué viaja en una petición HTTP?

- Ejemplo:
- GET /index.html HTTP/1.1
- Host: www.ejemplo.com
- User-Agent: Mozilla/5.0
- Accept: text/html

¿Qué responde el servidor?

- HTTP/1.1 200 OK
- Content-Type: text/html
- Content-Length: 1234
- `<html> <body>Hola Mundo</body> </html>`

Códigos de estado HTTP más comunes

- 200 OK: Todo salió bien
- 201 Created: Recurso creado correctamente
- 400 Bad Request: Error en la solicitud
- 401 Unauthorized: No autenticado
- 403 Forbidden: Acceso denegado
- 404 Not Found: Recurso no encontrado
- 500 Internal Server Error: Fallo del servidor

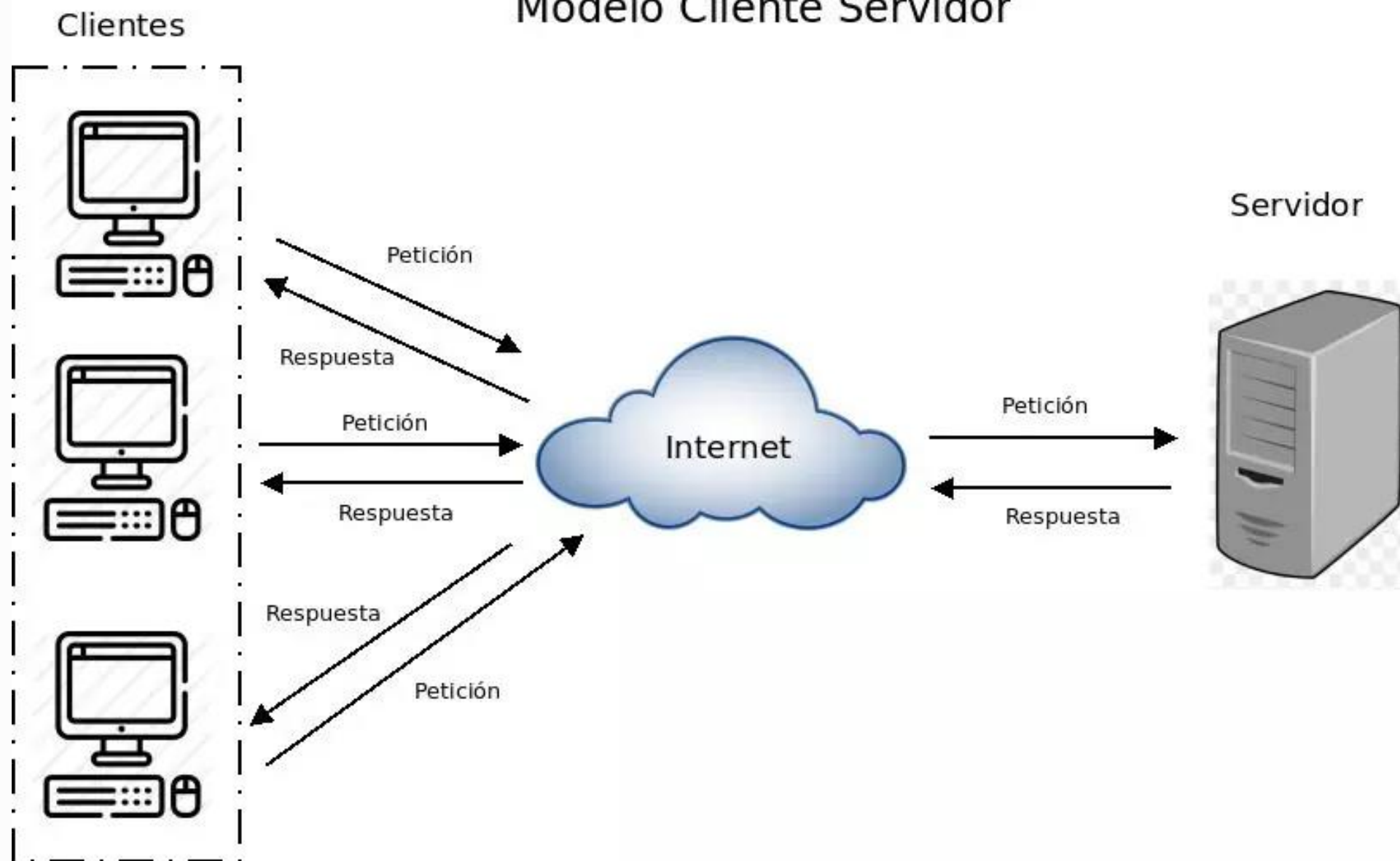
Actividad práctica con Postman

- 1. Hacer GET a <https://jsonplaceholder.typicode.com/posts/1>
- 2. Observar cabeceras, cuerpo y código de estado
- 3. Hacer POST y ver respuesta con código 201 Created
- GET POST PUT DELETE

Comparación: HTTP vs HTTPS

- HTTP: Comunicación en texto plano, puerto 80
- HTTPS: Comunicación cifrada (TLS), puerto 443
- HTTPS protege contraseñas, datos sensibles y evita ataques

Modelo Cliente Servidor



Introducción a GitHub

Control de versiones y colaboración profesional

¿Qué es GitHub?

- GitHub es una plataforma basada en Git para:
- • Control de versiones de proyectos
- • Colaboración en equipos de desarrollo
- • Integración con herramientas CI/CD
- • Publicación de proyectos (GitHub Pages)

Conceptos Clave de GitHub

- Términos principales:
 - Repositorio: Carpeta de tu proyecto con historial
 - Commit: Guardar un cambio con mensaje
 - Branch: Rama de desarrollo independiente
 - Pull Request: Solicitud de fusión de cambios
 - Fork: Copia de un repo para modificar

Flujo de Trabajo en GitHub

- Pasos básicos:
- 1. Clonar el repositorio con `git clone`
- 2. Crear rama nueva: `git checkout -b feature-x`
- 3. Hacer cambios y guardarlos con `commits`
- 4. Subir cambios: `git push origin feature-x`
- 5. Crear Pull Request y revisar cambios
- 6. Fusionar (`merge`) a la rama principal

Comandos Básicos

- `git init` → Inicializa un repositorio Git en la carpeta actual.
- `git clone <url>` → Clona un repositorio remoto a tu computadora.
- `git status` → Muestra el estado de los archivos (nuevos, modificados, preparados).
- `git add <archivo>` → Agrega un archivo al área de preparación (staging).
- `git add .` → Agrega todos los cambios al área de preparación.
- `git commit -m "mensaje"` → Guarda los cambios en el repositorio con un mensaje.
- `git log` → Muestra el historial de commits.
- `git diff` → Muestra diferencias entre archivos modificados y el último commit.

Git Ramas

- `git branch` → Lista todas las ramas y marca la actual con `*`.
- `git branch <nombre>` → Crea una nueva rama.
- `git checkout <nombre>` → Cambia a otra rama.
- `git checkout -b <nombre>` → Crea una nueva rama y cambia a ella.
- `git switch <nombre>` → Cambia a otra rama (forma moderna).
- `git switch -c <nombre>` → Crea y cambia a una nueva rama.
- `git merge <rama>` → Fusiona una rama en la actual.

Repositorio remoto

- `git branch` → Lista todas las ramas y marca la actual con `*`.
- `git branch <nombre>` → Crea una nueva rama.
- `git checkout <nombre>` → Cambia a otra rama.
- `git checkout -b <nombre>` → Crea una nueva rama y cambia a ella.
- `git switch <nombre>` → Cambia a otra rama (forma moderna).
- `git switch -c <nombre>` → Crea y cambia a una nueva rama.
- `git merge <rama>` → Fusiona una rama en la actual.

Actividad 4

Crear un repositorio Github

Actividad 5

Trabajo colaborativo

Actividad 6

Tarea Blog Personal

Herramientas de desarrollo

Paquetes Visual Studio Code

Extensiones en Visual Studio Code

- **Live Server** (Ritwick Dey)

Permite ver en tiempo real los cambios en el navegador al guardar.

- **Prettier - Code Formatter**

Autoformatea HTML, CSS y JS.

- **HTML CSS Support**

Autocompletado para atributos y estilos dentro del HTML.

- **Auto Rename Tag**

Si cambias una etiqueta `<div>` a `<section>`, automáticamente cambia la etiqueta de cierre.

- **IntelliSense for CSS class names**

Sugiere nombres de clases definidos en tu CSS al escribir en HTML.

- **Emmet (ya viene en VS Code)**

Permite escribir abreviado (`div.container>ul>li*3`) y expande a HTML completo.

- **Material Icon Theme o vscode-icons**

Mejora la visualización de archivos (muestra íconos según tipo de archivo).

- **W3C Web Validator** (opcional)

Valida directamente tu código con las reglas de W3C.

HTML y CSS

¿Qué es HTML semántico?

Uso de etiquetas que describen su propósito.

Mejora la estructura del documento.

Beneficios:

- Accesibilidad
- SEO (buscadores entienden mejor)
- Mantenibilidad

¿Qué es HTML semántico?

Metaetiquetas en <head>

Google detecta el <h1> como el tema principal de la página.

```
<> index.html > ...
```

```
1  <meta charset="UTF-8">
2  <meta name="description" content="Guía completa de HTML semántico y accesible para SEO.">
3  <meta name="keywords" content="HTML, semántica, SEO, accesibilidad">
4  <meta name="author" content="Profesor Vaixs">
5  |
```

Etiquetas semánticas principales

<header> → Encabezado de página o sección

<nav> → Navegación principal

<main> → Contenido principal

<article> → Contenido independiente

<section> → Agrupación temática

<aside> → Contenido complementario

<footer> → Pie de página

Accesibilidad (A11y)

Objetivo: hacer la web usable por todos.

Beneficia a:

Personas con discapacidad visual

Usuarios de lectores de pantalla

Personas en móviles con mala conexión

Buenas prácticas de accesibilidad

- Usar alt en imágenes.
- Formularios con `<label>` asociados.
- Texto claro y legible.
- Colores con contraste adecuado.
- Evitar “haz clic aquí” → usar enlaces descriptivos.

```
<form>  
  <label for="nombre">Nombre:</label>  
  <input type="text" id="nombre" name="nombre">  
</form>
```

ARIA (Accessible Rich Internet Applications)

- Atributos **ARIA** complementan HTML.

```
<nav aria-label="Menú principal">
  <ul>
    <li><a href="#">Inicio</a></li>
    <li><a href="#">Servicios</a></li>
  </ul>
</nav>
```

- Facilita navegación para usuarios con tecnologías asistivas.
- <https://validator.w3.org/>

Actividad 7

Corregir Blog Personal con HTML semántico y
Accesibilidad

Qué hace realmente loading="lazy"?

- El navegador NO carga la imagen inmediatamente al abrir la página.
- Solo la descarga cuando el usuario se acerca a verla (es decir, cuando está a punto de entrar en el área visible de la pantalla).
- Esto mejora el rendimiento y velocidad de carga inicial de la página, porque no gastas ancho de banda en imágenes que el usuario quizá nunca vea.

Validadores de accesibilidad:

- Despliegue su pagina
- Validadores de accesibilidad: WAVE o Lighthouse (en Chrome DevTools).
- <https://wave.webaim.org/>

CSS y SCSS

Validadores de accesibilidad:

CSS Básico y Buenas Prácticas

- Selectores: por etiqueta, clase, id y combinados.
- Box Model: margin, padding, border, width, height.
- Colores: rgba(), hsl(), variables CSS (--primary-color).
- Tipografía y unidades modernas: rem, em, %, vh, vw.

Buenas prácticas:

- Evitar usar IDs para estilos, preferir clases.
- Mantener consistencia (nombres descriptivos en clases).
- Usar reset o normalize.css para unificar estilos entre navegadores.

Validadores de accesibilidad:

CSS Básico y Buenas Prácticas

- Selectores: por etiqueta, clase, id y combinados.
- Box Model: margin, padding, border, width, height.
- Colores: rgba(), hsl(), variables CSS (--primary-color).
- Tipografía y unidades modernas: rem, em, %, vh, vw.

Buenas prácticas:

- Evitar usar IDs para estilos, preferir clases.
- Mantener consistencia (nombres descriptivos en clases).
- Usar reset o normalize.css para unificar estilos entre navegadores.

<https://cdnjs.cloudflare.com/ajax/libs/normalize/8.0.1/normalize.min.css>

Css Moderno

Flexbox → diseño en filas y columnas.

Grid → layouts complejos y responsivos.

Media queries → diseño responsive.

Animaciones y transiciones.

Pseudo-clases y pseudo-elementos: :hover, :nth-child(),
::before, ::after.

Css Moderno

Flexbox → diseño en filas y columnas.

```
<style>
.flex-demo{
  display:flex;                /* activa flex */
  gap:12px;                    /* espacio entre ítems */
  justify-content:space-between; /* eje x */
  align-items:center;          /* eje y */
  border:1px dashed #999; padding:8px;
}
.flex-demo > div{
  flex:1;                      /* ancho flexible */
  text-align:center; padding:16px; background:#eaf3ff;
}
</style>
```

Css Moderno

Grid → layouts complejos y responsivos.

```
<style>
.grid-demo{
  display:grid;
  grid-template-columns: 200px 1fr 200px;
  grid-template-areas:
    "header header header"
    "nav    main    aside"
    "footer footer footer";
  gap:12px; padding:12px; background:#f6f6f6;
}
.grid-demo > *{ background:#fff; padding:12px; border:1px solid #ddd; }
header{grid-area:header} nav{grid-area:nav}
main{grid-area:main} aside{grid-area:aside} footer{grid-area:footer}
</style>
```

Css Moderno

Media queries → diseño responsive. Animaciones y transiciones.

```
<style>  
.card{ padding:16px; background:#eaf3ff; }
```

```
/* ≥768px (tablets/desktop) */  
@media (min-width:768px){  
  .card{ background:#e8ffe8; width:60%; }  
}
```

```
/* ≥1024px */  
@media (min-width:1024px){  
  .card{ background:#fff3e6; width:40%; }  
}  
</style>
```

Css Moderno

Animaciones y transiciones.

```
<style>
/* Transición suave en hover */
.btn{
  padding:10px 16px; border:0; background:#2563eb; color:#fff;
  transition: transform .2s ease, background .2s ease;
}
.btn:hover{ transform: translateY(-2px) scale(1.02); background:#1e40af; }

/* Animación (keyframes) estilo “latido” */
.loader{
  width:18px; height:18px; margin-top:12px; border-radius:50%;
  background:#22c55e; animation:pulse 1s infinite;
}
@keyframes pulse{
  0%,100%{ transform:scale(1); opacity:1 }
  50%   { transform:scale(1.4); opacity:.65 }
}
</style>
```

SCSS Profesional

Buenas prácticas y organización de proyectos
grandes

¿Qué es SCSS?

- • Preprocesador de CSS (Sass)
- • Sintaxis tipo CSS, compilado a CSS
- • Permite variables, mixins, funciones, herencia
- • Mejora escalabilidad y mantenibilidad

Variables en SCSS

- `$primary: #0d6efd;`
- `$secondary: #6c757d;`
- `$font-stack: 'Roboto', sans-serif;`
- `$spacing-unit: 1rem;`

Anidación con moderación

- nav {
- background: \$primary;
- ul {
- li {
- a { color: white; &:hover { color: \$secondary; } }
- }
- }
- }

Mixins (reutilización de estilos)

- `@mixin flex-center($direction: row) {`
- `display: flex;`
- `justify-content: center;`
- `align-items: center;`
- `flex-direction: $direction;`
- `}`
- `.header { @include flex-center(column); height: 100vh; }`

Funciones personalizadas

- `@function px-to-rem($px, $base: 16) {`
- `@return ($px / $base) * 1rem;`
- `}`
- `dry`
- `h1 { font-size: px-to-rem(32); }`

Estructura profesional (7-1 Pattern)

- scss/
 - | — abstracts/ // variables, mixins, functions
 - | — base/ // reset, tipografía
 - | — components/ // botones, cards, navbars
 - | — layout/ // grid, header, footer
 - | — pages/ // estilos por página
 - | — themes/ // dark/light theme
 - | — vendors/ // librerías externas
 - | — main.scss // importa todo

Convenciones y uso moderno

- • Usar BEM (Block–Element–Modifier)
- .card {
- &__title { font-size: 1.5rem; }
- &--highlight { background: \$primary; }
- }
- • @use y @forward (en lugar de @import)

Buenas prácticas SCSS profesional

- • Modularizar código (7–1 pattern)
- • No abusar de anidación (>3 niveles)
- • Preferir mixins sobre extend
- • Documentar variables y mixins
- • Usar linters (stylelint)
- • Compilar y minificar en producción